KNUST Computer Engineering | Eliel Keelson

Application problems, essays, and 'worth thinking about' answers

Companion to the definitions guide. This booklet is how you SCORE on Friday: real scenarios, full essays (team project + SDGs; NFRs vs shareholders), and model answers to the Worth Thinking About boxes in Lectures 5 and 6.

Project lens: Regimens for Outstretched Arm Enterprise.
Exam habit: Situation -> Principle from the slide -> What we would do this week -> What we would not do.

How to write a scenario answer in 8-12 minutes

Lecturers who set 'worth thinking about' boxes are not hunting for a library name. They want to see that you can take a messy Ghanaian client situation and apply a named SE tool without panicking or inventing law. Use this skeleton every time: (1) Name the problem in one sentence. (2) Name the lecture tool (defect vs change, Act 690, GPL, MoSCoW, five-step pricing, four-step complaint). (3) Apply it to the facts. (4) State the next concrete action and who signs it. (5) Name one thing you will refuse to do (verbal yes, unpaid scope creep, shipping GPL into a closed product).

If the question is an essay, spend the first paragraph defining terms, the middle applying them to a project (use Regimens if they allow 'your team project'), and the last paragraph on limits and next phase. Do not list buzzwords. One worked example beats five slogans.

Essay A - Team project and the SDGs we hoped to contribute to

Write this as 500-800 words in the exam. Below is a full model you should shorten in your own voice. Do not claim we 'solved' the SDGs. The UN goals are global; a student inventory system can only contribute locally.

Opening

Software engineering is not only algorithms. It is a disciplined way to turn a messy human problem into a system that people can run, pay for, and maintain. Our COE 454 team, Team Regimens, was given a real Ghanaian micro-enterprise: Outstretched Arm Enterprise, a stationery shop that recorded stock and sales in a notebook. That notebook is a development problem disguised as stationery. When quantities are wrong, the shop over-orders or stock-outs. When prices change, profit on leftover stock becomes fiction. When the internet drops, a cloud-only app is as useful as a locked cupboard. We set out to replace informal memory with an offline-first sales and inventory system, and to do it as engineers: requirements, architecture, version control, and a client-facing demo, not as a weekend website.

What we built, in engineering language

The product, Regimens, lets an owner log in, manage products and categories, record a sale in three steps, see orders and reports, inspect gross profit, back up data, and change credentials. Functionally that is the shop's job. Non-functionally we treated security, usability at the counter, and reliability when the network fails as first-class, because a till that is pretty but offline-dead fails the client. Cost is stored in FIFO stock layers so leftover old stock keeps its arrival cost while the shelf price can move. That is an ethical as well as a technical choice: we refused to let later prices rewrite yesterday's economics. The stack is a modular Express API and Postgres on Render, with a React client on Vercel. We did not start with microservices. Lecture 2 told us not to. Seven people owned folders, reviewed pull requests, and under Ghana Copyright Act 690 we are joint authors until a written assignment says otherwise.

SDGs: contribution, not salvation

We mapped the work to a small set of Sustainable Development Goals so that 'impact' would not be a poster slogan. SDG 8 (Decent work and economic growth) is the closest fit. A shop that cannot see stock wastes labour recounting shelves and loses sales when a fast-moving pen runs out. Digitising that work is not 'creating Silicon Valley jobs'; it is reducing unpaid chaos for a trader and making the hours of an assistant more productive. We also took SDG 8 seriously on our side of the table: underpricing student work at GHS 1,500 for eight weeks is not decent work. Lecture 5's cost floor, deposit, and change requests are how engineers refuse to subsidise a

client with burnout.

SDG 9 (Industry, innovation and infrastructure) is about whether informal retail can use digital infrastructure at all. A notebook is infrastructure too, just a fragile one. Moving to a browser app hosted in-country-accessible cloud is a tiny piece of digital public infrastructure for one firm. The honest limit: free Render sleeps. If the till waits sixty seconds to wake, we have not delivered industrial-grade infrastructure. Keep-alive pings and a future paid instance are the next engineering steps, not a press release.

SDG 12 (Responsible consumption and production) is why FIFO costing is not a 'nice database trick'. Over-ordering because you cannot see cost layers produces dead stock: paper, plastic, transport, and money sitting in a cupboard. Accurate leftover cost and low-stock warnings help a shop buy what it will sell. We do not measure tonnes of waste avoided. We can say the system makes waste visible. SDG 4 (Quality education) sits in the background: stationery is an input to schools. A shop that stock-outs exercise books the week of exams is an education supply failure. We did not build a school platform. We built reliability into a supplier of school inputs.

SDG 1 (No poverty) is easy to over-claim. One inventory app will not lift a household out of poverty. It can reduce leakage from a micro-enterprise that is already the owner's livelihood. That is a contribution to economic resilience, not a census indicator. SDG 16 (Peace, justice and strong institutions) appears only weakly: dated sales records are better than a disputed notebook if GRA or a partner ever asks what happened. We are not a tax authority product.

Closing the essay

If asked 'did you achieve the SDGs?', the professional answer is: we selected a local problem that sits under 8, 9, 12 and partly 4; we built an MVP that makes those contributions testable; we did not confuse a course project with national policy. The engineering follow-through is phase two: staff roles so an assistant cannot see profit, receipts, a hosting retainer so the API does not sleep during shop hours, and a written contract if this becomes commercial. That is how SDG language survives contact with a Friday exam and a real client.

Essay B - Which NFRs we prioritise, and how we tell shareholders

Note the word shareholders. In a listed company they own equity. In our semester they are closer to stakeholders: the shop owner, staff, our team, KNUST assessors, and later any investor. Explain both. Non-functional requirements are quality attributes: security, performance, usability, reliability, maintainability, scalability, compliance. They compete. You cannot maximise all of them in an MVP. Architecture is the record of which ones you paid for.

The ranking for Outstretched Arm (defend this)

First, security appropriate to money-adjacent data. Sales totals and costs are not a hospital, but they are the owner's livelihood. JWT, hashed passwords, rate limits, CORS, parameterized SQL, owner-only backup, and a refused weak JWT_SECRET are the floor. I would tell a shareholder: 'If we ship without this, one leaked demo password or SQL injection ends the relationship and possibly GRA-grade embarrassment. Security is cheaper on day one than after a incident. We are not buying a pentest yet; we are buying the controls in the course security notes.'

Second, usability at the counter. A shop assistant serving a customer will not complete a five-page form. AppShell, cedis, low-stock colour, three-step sale, and protected routes are usability as a business requirement. Shareholder language: 'Adoption is the only ROI. An unused secure system has value zero. We traded fancy dashboards for a path the owner can finish in under a minute, because that is how the notebook actually dies.'

Third, reliability including offline and honest data. Dexie queue plus sync, and FIFO layers, are reliability of the business numbers, not just uptime. Shareholder language: 'A sale that vanishes when MTN drops, or profit that lies after a price change, will send them back to the notebook. We accept Render cold start as a known availability hole on the free tier, and we mitigate with health pings. We do not pretend five-nines.'

Fourth, maintainability: migrations, mock versus real data behind one interface, domain routes. That is how seven students do not destroy main. Shareholder language: 'Maintainability is the option to still have a product in December. It is cheaper than a rewrite when Raymond adds a table.'

We deliberately deprioritised: massive scalability (500 concurrent checkouts), pixel-perfect accessibility audit, MFA, microservices, native Android. Why? Lecture 5 scaling slide: scale when load is real. A Kotei-scale stationery shop is not Paystack. If a shareholder says 'but Twitter uses microservices', we use the building analogy: do not move the load-bearing wall because a blog post is fashionable.

How to speak to shareholders without sounding like a student

Translate NFR names into cash, risk, and time. Security = risk of loss and reputation. Usability = whether revenue is recorded at all. Reliability = whether the shop trusts the number enough to stop the notebook. Performance = seconds at the till; we set a qualitative target (interactive when warm) rather than fake 50ms SLAs we cannot measure. Scalability = optional until the API is CPU-bound. Compliance = we are not a bank; we still hash passwords and avoid storing secrets in git because GRA and clients can still ask how data is handled.

When they disagree, do not vote with feelings. Use MoSCoW and a one-page trade-off: 'If we spend this sprint on a mobile app (Could/Won't), we delay staff roles (Should) and keep the sleeping server (hurts reliability). Recommend: pay Render so it does not sleep, then receipts, then roles.' Ask them to pick with their wallet. That is professional, not rude.

Close the essay: NFRs are not a wishlist. They are a budget. We published our budget in ADRs and the security notes. Shareholders who want different priorities are requesting a change, which we price.

Lecture 5 - Worth thinking about, answered

Intellectual property

SCENARIO / QUESTION: If commercialised after graduation, who of the seven owns the code, in what proportion?

HOW TO TACKLE IT: Default under Act 690: we are joint authors. Ghanaian law does not magically split 14.28 percent each. Joint ownership means each co-author's rights depend on agreement; without a charter clause, any commercialisation can stall because one person can block or later claim. Tackle it this week: written split (equal, or weighted by backend/frontend/db/docs), rule for a member who leaves, and rule for client assignment. Do not 'remember we agreed in WhatsApp'.

SCENARIO / QUESTION: Have you checked every library licence? Could any block commercial use?

HOW TO TACKLE IT: Run npx license-checker --summary on frontend and backend before handover. Our intended stack is permissive (MIT/ISC/Apache). Dexie is Apache 2.0, still commercial-safe. Danger is a transitive GPL chart or a CC BY-NC icon. If GPL appears, we replace it before we sell. Attach the audit to acceptance.

SCENARIO / QUESTION: Client wants a full IP assignment. What do you need in return?

HOW TO TACKLE IT: Money (final payment before assignment bites), a licence back to reuse non-specific components, moral rights respected (names on the work), a support period defined so they cannot demand free

payroll later, and no unlimited indemnity. Assignment without payment is a gift.

SCENARIO / QUESTION: Six months later they sold the system to a third party without you.

HOW TO TACKLE IT: Read the contract. If IP assigned after payment, they may resell unless the contract forbade it. If no assignment, they likely only had a licence to use, not to sell; we send a written notice, then ADR. We do not threaten WhatsApp voice notes. We also ask whether our GPL hygiene was clean, or we have no moral high ground.

SCENARIO / QUESTION: Does the team charter say who owns what?

HOW TO TACKLE IT: If it only talks about branches and meeting times, it is incomplete. Add IP, payment split, and what happens if one person ghosts. That is configuration management for humans.

Pricing

SCENARIO / QUESTION: What is the cost floor for Regimens in GHS? Have you calculated it?

HOW TO TACKLE IT: Rough exam numbers (show the method, not fake precision): 7 people x 8 weeks x 8 hours x a student-professional rate, say GHS 40-80/hour, already tens of thousands if billed honestly. Add Render/Vercel, domain, revision rounds, QA 25 percent, deploy, docs, 30-day support, 20 percent uncertainty. A 'feel' price of GHS 1,500 is below cost floor. For a one-shop bespoke tool, a defensible quote might sit in the low thousands to low tens of thousands depending on whether we count course time as sunk. Say that out loud: course time is sunk for KNUST, but a commercial redo is not.

SCENARIO / QUESTION: Client says the price is too high. First move? What never to do?

HOW TO TACKLE IT: First: ask which part they object to, and re-explain value (hours saved, stock not lost). Offer a smaller MVP scope, not a secret 40 percent cut that you will resent. Never immediately slash the number to keep the peace. That trains them to negotiate with your self-esteem. Never start unpaid 'just this once'.

SCENARIO / QUESTION: What does one month of hosting cost, and does it inform pricing?

HOW TO TACKLE IT: Free tiers are not a business. Paid Render Postgres plus web plus a domain is a real monthly cost. SaaS or retainer must cover that plus your time when it breaks at 7pm. If we charge a one-time GHS 8,000 and host forever for free, we go broke at month four.

SCENARIO / QUESTION: When is a project unprofitable? Are you tracking hours vs estimate?

HOW TO TACKLE IT: When remaining work x rate exceeds remaining unpaid contract, or when change requests are absorbed as defects. We did not run industrial time sheets; in an exam admit the gap and say we would log hours against WBS next time.

SCENARIO / QUESTION: Same price for a Kotei sole trader and a multi-branch Accra firm?

HOW TO TACKLE IT: No. Value-based pricing: the Accra firm leaks more money when stock is wrong, needs more users, more SLA. Cost floor may be similar; willingness to pay is not. Also more compliance talk. Do not insult the Kotei trader; offer a smaller scope.

Contracts and payment

SCENARIO / QUESTION: Client wants to continue after graduation. What contract first?

HOW TO TACKLE IT: Scope of remaining work, payment (deposit, milestone, final), IP assignment or licence, change process, support window, termination, liability cap, hosting responsibility. Sign before new features. Electronic Transactions Act: email confirmation of terms can bind, so be careful in WhatsApp too.

SCENARIO / QUESTION: What in the MoU protects you, and what is missing vs a real contract?

HOW TO TACKLE IT: A student MoU often has goodwill and a problem statement. Missing: IP, payment, change pricing, liability, support end date. Do not pretend the MoU is Act 690-complete.

SCENARIO / QUESTION: Deposit paid, then they vanish. When do you stop?

HOW TO TACKLE IT: After written reminders citing the contract, stop new work when the deposit is earned against delivered artefacts. Do not keep coding into your nights hoping they return. Termination clause should say unused deposit handling.

SCENARIO / QUESTION: If they paid the team, how do we split?

HOW TO TACKLE IT: Agree in writing before the invoice: equal vs role-weighted, who pays mobile money fees, who invoices. Joint authors plus no split agreement is how teams explode.

SCENARIO / QUESTION: Earliest moment to mention payment terms?

HOW TO TACKLE IT: Before significant work, ideally at the proposal meeting. Deposit is not distrust. It is professional practice in Ghana (30-50 percent).

Change management

SCENARIO / QUESTION: Client in class tomorrow asks for a feature not in requirements. Exact words?

HOW TO TACKLE IT: 'Thank you, that is useful. It is not in the signed scope, so it is a change request, not a bug. I will assess database, API, and UI impact, send you a short written estimate, and we start only after you confirm by email. We will keep shipping the agreed MVP this week. We can put this first in phase two if you prefer not to pay now.'

SCENARIO / QUESTION: What is a defect for us, and what is the support period?

HOW TO TACKLE IT: Defect: login fails on valid credentials; sale does not decrement stock; profit uses current cost for old lots contrary to spec. Support: state 30 days of defect fixes after acceptance. New modules are not support.

SCENARIO / QUESTION: Cost of a change that touches DB, API, and frontend?

HOW TO TACKLE IT: FIFO layers were exactly that: migration 008, mock and real data, ProductManagement UI, demo docs. That is days, not an evening. Quote with uncertainty margin. Never say 'small change' until you walk the three layers.

SCENARIO / QUESTION: When is a change big enough to restart the contract?

HOW TO TACKLE IT: When it changes the product identity (payroll, e-commerce, multi-branch) or doubles remaining effort. Amend if it is a bounded extra; new contract if it is a new system.

SCENARIO / QUESTION: Verbal approval, then they refuse to pay. Evidence?

HOW TO TACKLE IT: You have almost none. That is why step 4 is written approval. After the fact: messages, commit dates, demo recordings help weakly. ADR and Small Claims want paper.

Winning clients

SCENARIO / QUESTION: Last client update: what would it say today?

HOW TO TACKLE IT: 'This week we finished FIFO cost layers so leftover stock keeps old cost. Next week we need you to change the demo password and confirm whether receipts are phase two. Blocker: Render free-tier spin-up on first request.'

SCENARIO / QUESTION: Angry call: system down. First thirty seconds?

HOW TO TACKLE IT: Acknowledge. Do not defend. 'I hear you, the till cannot wait. We are checking API health now. If Render slept, it should answer within a minute; if not, we will say so in ten minutes with a workaround: paper sales, then sync.' Then actually call back. Silence is the failure.

SCENARIO / QUESTION: One non-code thing this week to raise confidence?

HOW TO TACKLE IT: Send the written weekly update. Or a one-page user guide. Or show the backup restore once. Reliability theatre that is real.

SCENARIO / QUESTION: Handover: what is phase two and how priced?

HOW TO TACKLE IT: Staff roles (Should), receipts (Should), paid hosting so no sleep (reliability), UptimeRobot. Price as a package or T&M days. Could-haves: richer alerts. Won't: native Android this phase.

SCENARIO / QUESTION: Who else near KNUST might need this? Referral?

HOW TO TACKLE IT: Other stationers, campus print shops, pharmacy counters, chop bars with stock. Ask Outstretched Arm for an introduction after a week of stable use. Do not spam their competitors from the demo login.

Career and scaling (Lecture 5 tail)

SCENARIO / QUESTION: If you graduated tomorrow, is the portfolio enough?

HOW TO TACKLE IT: Regimens on GitHub with a clean README, live Vercel URL, and a paragraph on FIFO and offline is evidence. Hide nothing. Add the licence audit and a short ADR. That beats ten unfinished tutorials.

SCENARIO / QUESTION: One skill to deepen, and the plan?

HOW TO TACKLE IT: Pick one: backend (tests around sales transactions) or product thinking (pricing a retainer). Plan: one tested module per week, not a new framework.

SCENARIO / QUESTION: Do you know rates in Accra, Lagos, Upwork?

HOW TO TACKLE IT: If no, that is the answer they want you to fix. Research before you quote. Underpricing is a data problem plus fear.

SCENARIO / QUESTION: Freelance in six months: what must be in place?

HOW TO TACKLE IT: Written contract template, deposit rule, licence checker habit, separate money, tax awareness, a support window, and a portfolio URL. Not a new laptop.

SCENARIO / QUESTION: Five years: employed, freelance, founder?

HOW TO TACKLE IT: Any path is fine if you can explain the habit you are building now: writing agreements, communicating before panic, not starting with microservices. Founder without sales is an expensive hobby (slide).

SCENARIO / QUESTION: 500 simultaneous users tonight: what fails first?

HOW TO TACKLE IT: Render free web + single Postgres connections, then rate limiter, then unoptimised listSales loops. Vercel static files will be fine. Sequence: pay for always-on, indexes, then vertical, not Kubernetes.

SCENARIO / QUESTION: When is the project 'ready to scale'?

HOW TO TACKLE IT: When real shops hit slow queries and errors at rush hour, not when we are bored of the monolith. Define a number only with measurement: p95 login and sale times.

SCENARIO / QUESTION: A contract 3x larger than anything before: what to assess?

HOW TO TACKLE IT: Scope stability, deposit, whether we must hire, single-client revenue risk, legal (IP, liability), whether the tech (offline, FIFO) still fits. Ego is not an assessment.

SCENARIO / QUESTION: Busy vs growing?

HOW TO TACKLE IT: Busy is many unpaid extras. Growing is repeatable sales, retainers, documented process so a new person can help. Track which one your week looks like.

SCENARIO / QUESTION: Investor offers GHS 50,000 to scale. Spend order?

HOW TO TACKLE IT: 1) Always-on hosting and backups. 2) Automated tests around money. 3) Staff roles and receipts so more shops can use it. 4) Only then extra servers. Not a microservices rewrite. Not office space.

Lecture 6 - Worth thinking about, answered

SCENARIO / QUESTION: Have you run a licence audit this week? What licences?

HOW TO TACKLE IT: Command: npx license-checker --summary in frontend and backend. Expect MIT/ISC/Apache. Flag GPL/AGPL/SSPL immediately. Put the output in the handover pack. If you have not run it, say so and run it before you claim commercial-ready.

SCENARIO / QUESTION: Project uses a GPL library. Client asks if they can sell after you graduate. What do you tell them?

HOW TO TACKLE IT: If we distributed a product containing GPL code as closed source, copyleft can require the whole distributed work to be GPL. They might legally demand source; a competitor might too. We must replace the library with MIT/Apache, rebuild, and only then talk commercial sale. We do not hide this. Dual-licence the library vendor if they offer a paid proprietary licence (MySQL model).

SCENARIO / QUESTION: You want others to use a library free, but companies pay. Which model?

HOW TO TACKLE IT: Dual licensing: GPL (or AGPL if it is SaaS-shaped) for community, commercial licence for closed products. Qt and MySQL pattern. Need enough value that firms will pay to escape copyleft. MIT-only will not force payment.

SCENARIO / QUESTION: npm package switches MIT to AGPL next version, you are mid-build. Options?

HOW TO TACKLE IT: Pin the last MIT version (and hope no critical CVE). Or replace the dependency. Or accept AGPL and open the hosted source (usually death for a closed shop tool). Choose pin or replace, record in ADR, tell the client if it delays.

SCENARIO / QUESTION: Will handover include a list of OSS components and licences? Should it?

HOW TO TACKLE IT: Yes. Lecture 6 best practice: audit attached to MVP acceptance. Without it you cannot honestly assign IP or sell.

Real-life application problems (exam-style)

SCENARIO / QUESTION: Kumasi shop, no contract, cash paid, you later sell a similar system to a competitor. They threaten court.

HOW TO TACKLE IT: Slide: no contract, no IP transfer, you may be legally allowed, relationship destroyed. Tackle: do not gloat. Offer a narrow licence clarification, but next time always sign: client owns after final payment; you reuse generic modules. Reputation in Ghana's small industry is the real penalty.

SCENARIO / QUESTION: You quoted GHS 1,500, they add features for five weeks, you earn GHS 200/week.

HOW TO TACKLE IT: Classic underprice + no change process. Stop saying yes. Write change requests. Finish original scope. New work is GHS 150/day or similar. If already trapped, admit the lesson and do not repeat. Five-step pricing next time.

SCENARIO / QUESTION: Login works in spec but they want the colour of the brand-new church flyer.

HOW TO TACKLE IT: Change, not defect, unless the requirements named those colours. Assess, document, price (often small), written yes. If it is a one-hour CSS tweak you may include it as goodwill once, then freeze.

SCENARIO / QUESTION: Profit totals look 'wrong' because they changed cost yesterday and expect old

sales to move.

HOW TO TACKLE IT: If the spec said stamp unit_cost at sale time, this is working as designed, not a defect. Educate with the 2.50 vs 4.00 exercise-book numbers. If the spec said live recosting of history, then it is a defect. Requirements decide.

SCENARIO / QUESTION: Render is asleep, client misses a sale, they want a refund of the whole project.

HOW TO TACKLE IT: Complaint four-step. Own the NFR gap if we sold 'always on' on a free host. Explain hosting outages can be in the limitation of liability if contracted. Concrete step: paid instance + monitor, paper backup procedure. Do not disappear. Refund of entire build is not automatic; negotiate hosting SLA going forward.

SCENARIO / QUESTION: Assistant should not see profit, but you only have owner login. They call it a bug.

HOW TO TACKLE IT: If requirements never promised RBAC, it is a Should-have change. If the UI/UX guide promised staff vs owner, it may be a defect. Check the document, then either fix under support or price roles as phase two.

SCENARIO / QUESTION: You used a GPL chart in the profit page and already demoed.

HOW TO TACKLE IT: Stop further distribution talk. Replace Recharts-if-it-were-GPL with MIT (Recharts is actually MIT - in the exam still show the method). Rewrite the module. Tell the client before they sell. Two weeks of rework is cheaper than a lawyer.

SCENARIO / QUESTION: Designer used a CC BY-NC illustration on the login screen.

HOW TO TACKLE IT: NonCommercial. Cannot ship in a paid shop system without permission. Replace with CC0/MIT/original. CC is not for code but it still bites assets.

SCENARIO / QUESTION: You want to offer Regimens as hosted SaaS. Someone used AGPL in the stack.

HOW TO TACKLE IT: AGPL can trigger source offer to network users. For SaaS, AGPL is the 'AWS cannot take this quietly' licence, but it can force YOU to open your product. Remove AGPL deps or genuinely open the service. SSPL is even harsher and not OSI.

SCENARIO / QUESTION: MongoDB-style SSPL database as a dependency.

HOW TO TACKLE IT: Not OSI open source. Read production restrictions. Prefer Postgres (what we use) under a known licence. Do not 'just npm install' a source-available engine into a client deliverable.

SCENARIO / QUESTION: Team member leaves and takes the GitHub repo private, claims sole ownership.

HOW TO TACKLE IT: Joint authorship. Demand access restoration. Charter should have had org ownership (Logical883/regimens or a KNUST org). Social: do not escalate on Twitter first. Written notice, then department/ADR.

SCENARIO / QUESTION: Client records you on a phone agreeing to 'just add payroll by Sunday'.

HOW TO TACKLE IT: Still get email confirmation of scope and price before coding. A video of a maybe is not a change request. Repeat the five steps on the record.

SCENARIO / QUESTION: ECG outage during a sale. Is that your defect?

HOW TO TACKLE IT: Limitation of liability / environmental. Offline queue helps when the PHONE still has battery and local browser storage. You cannot fix ECG. Explain calmly with the paper-fallback procedure.

SCENARIO / QUESTION: They want the source after paying GHS 500 with no assignment clause.

HOW TO TACKLE IT: Default: you still own copyright. They paid for a copy/use unless the contract said assignment. Negotiate assignment as a paid extra, or deliver compiled/hosted use only. Be fair: if you implied

they would own it, you have a relationship problem even if the law is on your side.

Short scenario essays (15-20 line answers)

1. Scope freeze without losing the client

Once requirements are signed, that version is frozen for the phase. New ideas go to a backlog. Say: 'Great idea. Logged. Won't Have this phase means we will discuss it first in phase two, not never.' The amateur says yes and misses the demo. The amateur also says no forever and loses the referral. MoSCoW is the polite fence.

2. Fixed-price vs T&M for a shop that 'will know what they want as they see it'

That sentence is a T&M or a tiny fixed MVP plus paid changes. A single fixed price for an evolving dream puts all overrun on the students. If they insist on fixed, freeze a brutal MVP (login, product, sale, profit) and put the rest in a rate card.

3. Moral rights after they own economic rights

They can own copies and resale if assigned. They cannot force you to take your names off the work or to accept a mangled fork that harms your reputation without your objection rights. Practical: keep AUTHORS file. Do not sign illegal waivers as if they were valid under the slide's reading of Ghanaian moral rights.

4. Explaining copyleft to a trader who sells pens

Do not say 'viral licence'. Say: 'Some free code is a gift with a string: if we put it inside your shop app and give you the app, the law may force us to give the same freedoms to anyone, including your rival. We only use gifts without that string (MIT), or we pay the author for a private licence.'

5. Dual licensing your FIFO helper later

If stockLayers.js became a popular npm library, you could offer GPL to OSS and a commercial licence to closed ERPs. Only works if the helper is painful to rewrite. Until then, MIT it and do not fantasise about Oracle-style MySQL revenue.

6. Proactive communication as an NFR of the engagement

Lecture 5: clients get anxious in silence. A weekly two-line update is part of quality, like uptime. WhatsApp for brief, email for money and scope. The team that clients rave about is often the reliable communicator, not the cleverest reducer.

7. Turning handover into a retainer

MVP meeting is not goodbye. Present Could/Won't as a priced roadmap. Offer hosting+support monthly. Ask for a referral. That is SDG 8 for you: recurring decent work rather than one exhausted unpaid encore.

Oral cheat - 40 seconds each

- Act 690: code is a literary work; we own until assignment; seven joint authors; moral rights stay.
- Defect vs change: is it in the requirements? Yes = fix in support. No = assess, write, price, email yes, then build.
- MoSCoW: Must / Should / Could / Won't this phase.
- Price: cost floor, market, value, position, ask; add QA, deploy, docs, support, 15-20 percent.

- Deposit 30-50 percent. Unpaid: remind, demand, ADR, Small Claims under 10k.
- Complaint: hear, own, explain hosting, next step with a clock.
- MIT: keep notice, sell. GPL: distribute then whole work GPL. AGPL: network use can force source.
- Scale product: measure pain, optimise, bigger box, then extract. Scale business: demand > quality capacity.
- SDGs we claim: 8, 9, 12, a little 4. We contribute; we do not 'complete' them.
- NFR order for this shop: security floor, counter usability, offline+honest profit, maintainability; not k8s.

Full mock exam essay - 'Professional practice in a student client project'

Use this as a third essay you can adapt if the paper is open-ended. Aim for four to five sides in the real booklet; here is the argument in full.

A student team that ships features but cannot explain money, ownership, or licences has only completed half of Software Engineering II. Lecture 5 argues that what separates a good developer from a great one is rarely a new framework. It is knowing who owns the work, pricing to value, putting agreements in writing, communicating before the client asks, handling complaints without losing composure, and turning one project into the next conversation. Regimens forced us to practise those muscles on a stationery shop, not on a toy todo list.

Ownership came first. Seven people committed to one GitHub repository. Under the Copyright Act 2005 (Act 690), software is a literary work and copyright exists at creation. We are joint authors. Payment by a client, or a lecturer's deadline, does not move economic rights unless a written assignment says so. That fact is uncomfortable in a course where 'the client' is also a learning partner. The professional move is not to hide it. It is to write a one-page addendum: joint ownership among the seven until assignment; if Outstretched Arm later pays for commercial use, assignment happens after confirmed final payment; we retain moral rights and a licence to reuse generic modules such as auth middleware and the FIFO helper. Without that paragraph, graduation week becomes a fight about who can sell a fork.

Pricing is the second practice. The slide's common mistake is a GHS 1,500 quote that explodes into eight weeks of unpaid kindness. The five-step method stops that: cost floor (time, hosting, QA, support, uncertainty), market comparables, value to the shop (lost sales from stock-outs, hours spent on the notebook), positioning, and asking what they expected to pay. Even if KNUST forbids collecting money, we should still compute the floor so we know what 'too cheap' means. A retainer for hosting is not greed. The free Render instance sleeping mid-sale is an NFR failure funded by a price of zero.

Contracts and change control are how we stay friends. Fixed-price fits a frozen MVP: login, products, sale, profit. Time-and-materials fits 'we will know when we see it'. The most important distinction in the lecture is defect versus change. If the requirements said stamped historical cost, a client who wants history rewritten is asking for a change. If login fails on valid credentials, that is a defect inside the support window. The five steps (assess, document, price, written approval, build) are slower than WhatsApp yes. They are also how you survive Small Claims with evidence. Electronic Transactions Act 2008 means a sloppy email can bind you, so we should be as careful in writing as in SQL.

Licensing is architecture. MIT, ISC, and Apache 2.0 let a closed shop product exist if we keep notices. GPL can force the whole distributed product open. AGPL can do the same for a network service. Creative Commons NonCommercial cannot dress the login screen of a paid system. The application problem is not 'memorise 1983'. It is running license-checker before handover and recording the choice in an ADR. Shipping Recharts is fine because it is MIT; shipping a random GPL gauge because it looked nice is how a second client never happens.

Client relations sit on top. Weekly two-line updates. Four-step complaints. Scope freeze with MoSCoW so 'Won't this phase' is a diary entry, not an insult. At handover we present phase two (roles, receipts, paid hosting) and

ask for a referral. That is how a course project becomes a career artefact instead of a forgotten ZIP file.

Conclusion: professional practice is the runtime of software engineering. Code is the binary. If we can explain Act 690, a cost floor, a change request, and a GPL incident using Outstretched Arm as the example, we are ready for Friday. If we can only list React and Postgres, we are still in Software Engineering I.

Worked numbers - FIFO as a client education scenario

The owner bought 50 exercise books at GHS 2.50 each and later restocked 5 at GHS 4.00, then raised the shelf price to GHS 6. They ask why profit on the next 50 sales is not (6-4) x 50. Application answer: leftover old stock is a layer of 50 at cost 2.50. FIFO sells those first at the NEW shelf price 6. Line profit is 6-2.50 = 3.50 each, not 2.00. After 50 units, the five new books yield 6-4 = 2.00 each. If we had overwritten cost_price on the product row, every remaining book would suddenly look like a 4.00 book and we would have lied about old stock. That lie is both an NFR (integrity of analytics) and an ethics issue. If they insist on recosting history, that is a change request that would break stamped sale_items on purpose. We would price it and warn that GRA-style questions become harder.

Worked numbers - a defensible quote sketch

Suppose we commercially rebuild in 4 weeks with 2 people at GHS 60/hour, 30 hours/week: 2 x 4 x 30 x 60 = GHS 14,400 labour. QA 25 percent = 3,600. Hosting year-1 prepaid GHS 800. Three revision rounds included; extra GHS 150/day. Support 30 days bundled. Uncertainty 20 percent on labour+QA. Cost floor around GHS 14,400 + 3,600 + 800 + 0.2*(18,000) = about GHS 22,400. Market for a tiny shop tool might not bear that, so we cut SCOPE (no payroll, no Android) rather than lying about hours. Value argument: if stock-outs cost them GHS 400/month, a year is GHS 4,800 in recovered sales plus hours not spent on the notebook. Maybe we offer GHS 8,000 one-time plus GHS 200/month hosting for a reduced MVP. The exam mark is the METHOD and the refusal to quote GHS 1,500. Do not treat these figures as official invoices.

More application problems

SCENARIO / QUESTION: A teammate commits a .env with JWT_SECRET to main. Client repo is private. Fine?

HOW TO TACKLE IT: No. Private is not a control. Rotate the secret, purge if possible, add gitignore (already there), and treat it as a security incident in the handover notes. Limitation of liability does not excuse leaking secrets. This is ethics plus configuration management.

SCENARIO / QUESTION: Owner wants WhatsApp-only 'contract'. No email.

HOW TO TACKLE IT: Export the chat, restate terms in a single message they reply 'I agree' to, and still try for a PDF. ETA 2008 can treat electronic assent as binding, which cuts both ways. Better a short signed paper at the shop than a voice note.

SCENARIO / QUESTION: They ask you to scrape a competitor's prices into the app.

HOW TO TACKLE IT: Refuse. Professionalism and possibly computer misuse. Not in scope. Not an NFR. Offer manual price entry. Ethics question disguised as a feature.

SCENARIO / QUESTION: Investor wants you to relicense the whole repo GPL to look 'open'.

HOW TO TACKLE IT: GPL on our code is a team IP decision of all joint authors. It also interacts with what the client thinks they bought. Do not flip the licence to impress a demo day. If we go OSS, MIT/Apache is the usual

shop-tool choice unless we want copyleft.

SCENARIO / QUESTION: HashiCorp-style BUSL on a library you need.

HOW TO TACKLE IT: Source-available: no production commercial use for a period, then it flips to OSS. Read the change date. If our client needs production now, pick an MIT alternative. Do not assume 'GitHub = free to sell'.

SCENARIO / QUESTION: Client's cousin 'who knows computers' wants admin on Render.

HOW TO TACKLE IT: Access control is an NFR. Give them a read-only status page or wait for staff roles. Shared cloud consoles are how backups vanish. Put it in the contract who the operator is.

SCENARIO / QUESTION: You promised 'offline' but IndexedDB clears when they use Safari private mode.

HOW TO TACKLE IT: Usability/reliability gap. Document the constraint. Defect if we promised any browser including private. Change if we specified Chrome installed. Educate; consider a visible warning.

SCENARIO / QUESTION: GRA asks the owner for records. They ask you to fabricate missing days.

HOW TO TACKLE IT: Refuse. Integrity of records is why we stamp sales. Offer export of what exists. Fabrication is not a change request we price.

SCENARIO / QUESTION: Another KNUST team used our README verbatim in their report.

HOW TO TACKLE IT: Copyright in documentation too. CC-BY if we chose it; otherwise all rights reserved. Course misconduct is a department issue. Keep our README original enough that copying is obvious.

SCENARIO / QUESTION: You want to demo on Friday but JWT expired and seed user was deleted on Render.

HOW TO TACKLE IT: Operational reliability. Have a documented seed, health check, and a local mock fallback (USE_MOCK_DB) for the oral if production is dead. Communicate the fallback; do not pretend live multi-tenant scale.

Stakeholder map - who hears which NFR

Owner: usability, honest profit, backup. Assistant: speed of sale, few buttons, no finance screens. Us (team): maintainability, licence cleanliness, IP. Lecturer: evidence of SDLC and professional practice. Future investor: scalability story that starts with measurement. Hosting vendor: availability SLO you actually pay for. Explaining NFRs is matching the audience. Do not lecture the owner on AGPL. Do not lecture the examiner only on button colour.

A one-minute shareholder (investor) pitch: 'We prioritise security and data integrity because this is the shop's ledger. We prioritise counter usability because unrecorded sales are lost revenue. We postpone horizontal scale until a paid instance is saturated. The biggest current availability risk is a sleeping free server, which is a pricing problem as much as a Kubernetes problem. Here is the invoice for always-on hosting. Here is what we will not build until you fund it.'

Licence spectrum memory table in sentences

Unlicence/CC0: public domain style, almost no conditions. MIT/ISC: do anything, keep copyright notice, safest default for npm. BSD 2/3: like MIT; 3-clause adds no-endorsement. Apache 2.0: MIT plus patent grant and note your modifications if you distribute them; compatible with GPLv3 not GPLv2. MPL 2.0: file-level copyleft; rest of app can stay closed. LGPL: link from proprietary, share changes if you modify the library (glibc reason Linux apps can stay closed). GPLv2/v3: distribute the combination, the combination is GPL. AGPLv3: even remote

network users can trigger source obligations. CC-BY: credit. CC-BY-SA: share-alike. CC-BY-NC: no commercial. CC-BY-ND: no derivatives. SSPL: not OSI, service providers must open a huge stack. BUSL: not production commercial until it converts. Dual licence: pick copyleft or pay. Exam trap: 'open source on GitHub' is not a licence.

History anchors if they love dates: GNU 1983, GPLv1 1989, Linux GPLv2 1991, term open source and OSI 1998, Apache 2.0 2004, GPLv3 and AGPLv3 2007, Mongo SSPL 2018, later BUSL debates. Use at most two dates in an essay so you do not look like a timeline bot.

Combined killer scenario (write this if they give one long question)

Facts: After graduation the owner pays GHS 6,000 cash, no contract. You used an AGPL reporting widget. They want source, want to resell to two other stationers, and want you to add payroll by next week for free because 'we are family now'. They also complain profit is wrong after a price change. Tackle in order: (1) Stop coding. (2) AGPL: do not resell until the widget is gone; tell them the legal risk in plain language. (3) Act 690: cash did not assign IP; resale is a new commercial conversation. (4) Payroll is a change, not a defect, not family law. (5) Profit: show FIFO numbers; likely working as specified. (6) Write a one-page contract: assignment after a proper invoice, licence audit attached, change rate, 30-day defect support, hosting retainer. (7) Four-step complaint on the emotion, then the paper. If you do this in order in the exam, you have used Lectures 5 and 6 as an integrated professional system, which is the point of the course.

Sample 30-minute paper (practise with a timer)

Question 1 (essay, 15 min): 'Using your team project, discuss how software engineering can contribute to selected SDGs. Be critical.' Mark scheme in your head: define SE, name the shop problem, pick three SDGs with one system feature each, one limitation, no saviour language. Question 2 (essay, 10 min): 'Not all quality attributes can be maximised. Justify the NFR priorities of a system and explain how you would defend them to shareholders.' Mark scheme: define NFR, rank, money/risk translation, one item you refuse to fund yet. Question 3 (scenario, 5 min): 'A client who paid cash with no contract wants source code and the right to resell. Advise both parties.' Mark scheme: Act 690 default, relationship vs law, what a contract should have said, GPL hygiene if relevant.

Self-mark: Did I name a lecture tool? Did I use Regimens facts? Did I admit a gap? If all three, you pass the application paper even with imperfect English. If you only listed SDG numbers without a feature, you fail the 'application-wise' test.

Timing drill: close this PDF, open a blank page, write Question 1 in 12 minutes, check against Essay A. Then Question 2 against Essay B. Do not copy sentences. Copy structure. That is the last useful hour before you sleep.

Glossary of tools (say the definition, then the shop)

SDLC: stages from problem to maintenance. We used them as a loop, not a single waterfall pass. Functional requirement: a behaviour such as record sale. Non-functional: a quality such as works offline. Architecture: load-bearing decisions; ours is a modular monolith API plus a React SPA. ADR: why we chose Postgres over a spreadsheet or Firebase. MoSCoW: Must, Should, Could, Won't-this-phase. Defect: breaks signed spec. Change: new spec. Cost floor: below this you lose money. Value-based price: what the leak costs the shop. Fixed-price: we hold overrun risk. Time-and-materials: the client holds it. IP assignment: economic rights move in

writing. Moral rights: name and reputation stay. Permissive licence: MIT, keep the notice. Strong copyleft: GPL if you distribute. AGPL: network use can trigger source. Dual licence: pay to keep it closed, or take GPL to share. Scope freeze: new ideas go to a backlog, not a silent yes. Retainer: monthly hosting plus defect fixes. ADR Act 2010: faster than court. Small Claims: amounts under GHS 10,000. Electronic Transactions Act 2008: electronic assent can bind. Shareholder: equity owner. Stakeholder: anyone affected. If the paper says shareholders, still translate NFRs into cash, risk, and time.

Repeat those sentences until they feel boring. Boring means you own them. Then attach one Regimens fact and you have an answer.

If the oral stalls - backup stories

Story 1, the notebook: they lost a day's sales when rain wet the page. That is why sync_status and backups exist. Story 2, the sleeping server: first Vercel login spun for a minute; we added a 30-second health ping and documented UptimeRobot. Availability is paid for. Story 3, the price change: old books at 2.50, new at 4.00; FIFO is how we keep SDG 12 and the owner honest at the same time. Story 4, seven authors: nobody 'owns main' personally; the org repo does. Story 5, MoSCoW after demo: staff roles Should, Android Won't, cost layers Must because they already change prices. If you can tell these five without notes, you can fill an oral even when the question is oddly worded.

Last reminder: the examiner has seen teams claim microservices, claim they solved poverty, and claim MIT and GPL are 'the same because both are free'. Do the opposite. Be precise, be local, be honest about gaps. That is application-wise software engineering.

Night-before essay plan

If the paper says 'discuss your project and SDGs': pitch, three SDGs with one evidence each, one honest limit (Render sleep or no staff roles), phase two. If it says 'NFRs and stakeholders': define NFR, rank four, translate to money/risk, one deprioritised item with the scaling slide. If it is a scenario: skeleton of five sentences using a named tool. Write legibly. Underline the tool name once. That is how application papers are marked.

This booklet is longer than an exam booklet on purpose. Do not memorise paragraphs. Memorise the tools and one Regimens story for each. Good luck on Friday.